

Cash-Flow Forecaster

How I planned and approached the app

The first step was deciding on the method I'd use to solve the problem.

The core mechanic of this app rests on a few key assumptions. It only accounts for recurring income and bills (weekly, monthly, annual), so a balance-checkpoint field always has to be available to absorb everything else — food, random spending, anything not modeled as a recurring item.

Once I had a well-reasoned structure, I asked Claude to generate a `CLAUDE.md` file, mainly to keep my decision making anchored to that initial structure. Changes still happened along the way, but the file kept my agentic coding assistant from drifting off it without a reason.

Why I chose these tools, frameworks, and libraries

Next.js was chosen because Vercel makes it trivial to deploy straight from a GitHub repo. I wanted a working, physical proof of the idea, since this brief is more of a data-visualization problem than a complex algorithmic one.

TypeScript was chosen for type safety on input objects, especially useful once the app is expected to hold many different item objects over time.

Main technical decisions and reasoning

A few technical decisions stand out.

The first was around sorting. Initially, I planned to take every billing item, sort all of them, and walk the balance across that full sorted list. That turned out to be unnecessary once I decided to only forecast the current month. The only payments that matter are the ones landing inside that window.

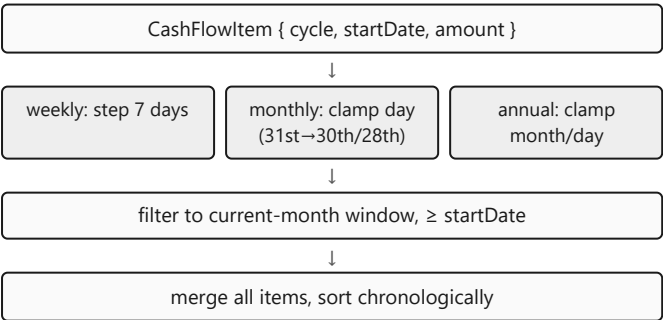
Since each item already carries a start date, I introduced a separate Occurrence list that stores only the expanded, exact dates each recurring item falls on within the window. This avoids generating objects for an item's entire lifetime and keeps everything scoped to roughly a 30-day period.

Sorting itself only matters from a visualization standpoint — every occurrence still has to be checked regardless of order, since the balance walk is a single pass through the whole set either way.

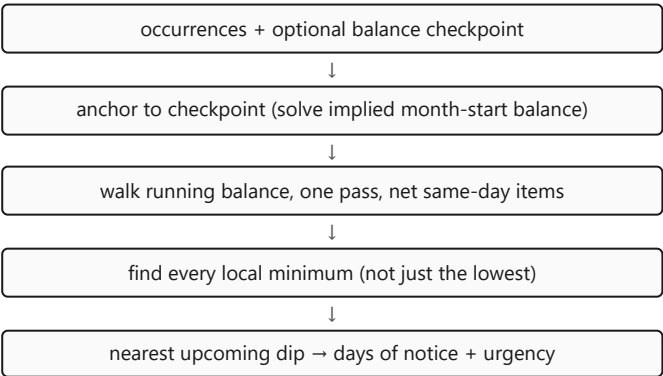
One rejected decision. The reminder mechanic was ignored although the spec mentioned it, this is because I'm specifically letting people access this on a website. If it were to be a phone app, a notification reminder 7 days ahead of a dip would be perfect, but I opted for a monthly visualization graph instead.

Key flows

Recurring item → sorted occurrence list



Balance walk → "when it gets tight"



Architecture overview

UI (route-based tabs: Reports, Calendar, List, Inputs) calls one shared data function that assembles items, occurrences, the balance walk, and the lead-time insight. That function sits on top of pure, framework-free logic (occurrence generation, balance walk, local minima), which reads from an in-memory server store — no database, intentional for this scope.

Layer	Choice
Framework	Next.js (App Router, Server Actions, Server Components)
Language	TypeScript
UI components	shadcn/ui (Radix primitives + Tailwind)
Charts	Recharts
Testing	Vitest — unit tests on occurrence generation and forecast logic
Hosting	Vercel, deployed directly from the GitHub repo

Where I used AI, and how I checked its output

AI (Claude / Claude Code) was used throughout the whole build — brainstorming, writing code, checking my own reasoning. Being upfront, I didn't write a single line of code myself in this assessment. That doesn't mean I accepted every output though.

Rather than prompting "build feature X," I broke things down into small, specific questions like "generate a type-safe in-memory store for CashFlowItem," so each new feature stayed small enough to actually review, rather than trusting one large diff wholesale.

Core functionality was checked more carefully. I read through the logic to understand why Claude made the decisions it did, and pushed back or asked for a rework whenever something felt more complex than the situation called for.

For the UI, I deliberately let AI drive without much oversight — design wasn't the point of this assessment. I did make sure it was grounded in something real (Mobbin references and an actual shadcn component library) so the result stayed visually consistent rather than improvised.